

Introducción a la computación evolutiva

Integrantes:

Ignacio Gonzalez Barboza
Facundo Bonanna

Profesora:

Virginia Yannibelli

Correos de contacto de miembros:

igonzalezbarboza@alumnos.exa.unicen.edu.ar
fbonanna@alumnos.exa.unicen.edu.ar

Correos de contacto de la cátedra:

icompevol@catedras.exa.unicen.edu.ar

Repositorio de Github con el código:

[Link](#)

Índice

Índice	2
1. Introducción	2
2. Metodología	3
2.1 Lenguaje	3
2.2 Representación	3
2.3 Población	3
2.4 Cruce	3
2.5 Mutación	3
2.6 Selección de Padres	4
2.7 Selección de Sobrevivientes	4
2.8 Evaluación	5
2.9 Condición de Terminación	5
4. Experimentos Computacionales	5
Problema P43	5
Problema BR17	6
5. Resultados y conclusiones de cada problema	6
Problema P43	6
Problema BR17	7
6. Conclusiones generales	7
7. Bibliografía	8

1. Introducción

Para el trabajo el grupo tuvo que diseñar e implementar un algoritmo evolutivo para obtener una solución cercana a la óptima y la óptima del Problema del Viajante. En la aplicación de software se recibe un archivo con el formato de .atsp, el cuál contiene una descripción breve del problema (como el nombre, dimensiones, entre otros) y los costos entre cada par de ciudades. Dentro de las características del problema, tuvimos en cuenta que los caminos no tienen el mismo costo de ida que a la vuelta, lo cuál fue importante para la toma de decisiones. Para el desarrollo del programa se utilizó el contenido provisto por la cátedra y fue utilizado el lenguaje de programación de Java. Finalmente, el programa debe permitir configurar

parámetros (como los métodos de cruce, selección, probabilidad de mutación, etc) y presentar un archivo que resuma los resultados (mejor fitness en cada generación, como está compuesta la mejor solución, su valor de fitness, entre otros) del experimento al usuario.

2. Metodología

2.1 Lenguaje

Utilizamos el lenguaje de Java debido al paradigma de la Programación Orientada a Objetos. Creemos que abstraer el problema en clases, como “Población”, “Solución”, “Operador de Cruce”, etc, facilita la comprensión de los conceptos vistos en clase.

2.2 Representación

El Problema del Viajante requiere que la solución sea una secuencia de ciudades a recorrer, respetando que una ciudad no sea visitada 2 veces. Por lo tanto va a ser representado por un arreglo de largo n (Siendo n la cantidad de ciudades) y que cumpla las características de una permutación. Es importante destacar que una misma solución puede ser representada de n veces distintas, ya que una misma secuencia puede realizarse partiendo de n ciudades distintas.

2.3 Población

La población va a ser representada por una lista de soluciones. Aquí decidimos utilizar la librería de `java.util.list`, ya que métodos como “sort”, “size”, “isEmpty”, entre otros, restaron complejidad a la programación, permitiendo seguir nuestro objetivo de abstraer lo mayor posible.

2.4 Cruce

Según la cátedra, el cruce es “exploratorio” y eso se pudo ver en la práctica al implementar el algoritmo. El campo de soluciones está repleto de mínimos, ya sean óptimos o locales. El uso de operadores de cruce consiste en dar dos soluciones, extraer sub secuencias de ambas y combinarlas para obtener una tercera solución. Esto es clave ya que mientras mayor sea la variabilidad, menor va a ser la probabilidad de estancamiento.

Ejemplos de operadores de cruce son:

Orden: este operador consiste en sacar una subsecuencia de un padre y se completa con las ciudades que quedan visitar siguiendo el orden del otro padre. Esto genera variación, ya que la primera subsecuencia se respeta, pero puede generar demasiada variación y es contraproducente.

Arcos: Este operador fue el más complejo pero logra que se mantengan o prioricen los arcos de ambos padres. Esto genera la variabilidad que se necesita, sin lograr una solución que está muy lejos de su óptimo local.

2.5 Mutación

Al contrario del cruce, la mutación puede ser descrita como “explotadora”. Esto se debe a que se busca llegar a nuevas soluciones cercanas, permitiendo hacer pequeños cambios y obtener la mejor solución en la cercanía. Este operador ayuda a obtener los óptimos locales e incluso en alguna generación, la secuencia óptima.

Ejemplos de operadores de mutación son:

Inversión: se seleccionan dos extremos al azar y se invierten las ciudades entre medio. En otros problemas, este operador generaría poca variación, debido a que se rompen solo 2 enlaces, pero como la matriz tiene distintos costos de ida y de vuelta, puede generar mucha variación.

Intercambio: este operador intercambia dos ciudades entre sí. Es muy sencillo y romperá con seguridad 4 links. Teniendo en cuenta lo dicho previamente, aunque genere variación, será más constante y menos “destructiva” que algunas mutaciones por inversión.

2.6 Selección de Padres

Luego de ordenar de menor a mayor, según su fitness, la población, es clave elegir qué soluciones serán utilizadas para generar nuevas. Investigando por más información sobre la selección de padres, encontramos que; Según Freire (2024), “los rasgos más beneficiosos se acumulan en la población a lo largo del tiempo, ya que los individuos que los poseen tienen más éxito en la competencia por los recursos” (párr. 7).

Algunos ejemplos de métodos para seleccionar a los padres son:

Por Ruleta: se genera una ruleta donde un mayor fitness a comparación del promedio, permite tener mayores posibilidades de ser elegido.

Torneo: se determina un tamaño k para el torneo y compiten k soluciones al azar y sobrevive la de menor fitness. Es importante que el k sea chico, ya que mantendrá la diversidad de soluciones y no llegará a una convergencia temprana. Otro aspecto a tener en cuenta es si las soluciones elegidas siguen siendo tenidas en cuenta para próximos torneos.

2.7 Selección de Sobrevivientes

Este mecanismo de selección consiste en elegir qué soluciones se preservarán en la próxima iteración. Los mecanismos se dividen en dos grandes tipos; por edad y por fitness.

Algunos ejemplos son:

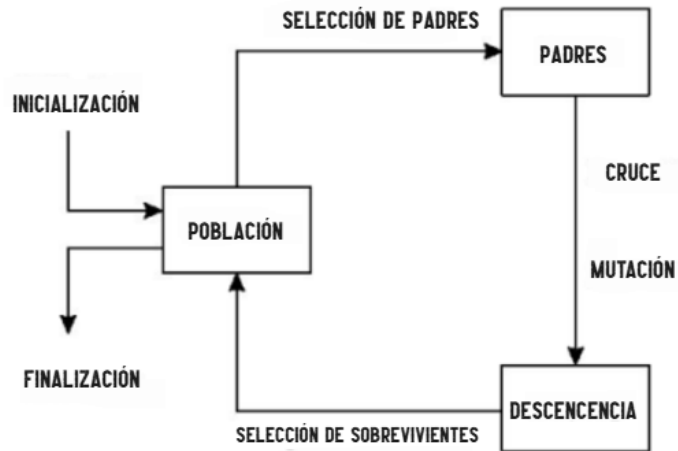
Reemplazo Padres por hijos: este es un mecanismo que considera la edad a la hora de elegir a los sobrevivientes. Una desventaja es que cuando se reemplazan los padres, se pueden perder soluciones muy buenas.

Steady State: este mecanismo considera el fitness sobre la edad. Se intercambian las n soluciones peores de la iteración anterior por las n soluciones mejores de la nueva iteración. Steady State tiene un problema que es que puede llegar

a una convergencia prematura rápidamente, esto se resuelve con poblaciones grandes o políticas que preserven la diversidad.

2.8 Evaluación

En el experimento seguimos el esquema dado por la cátedra:



2.9 Condición de Terminación

La condición de corte determina cuántas iteraciones se van a realizar. Utilizamos un contador para realizar las n iteraciones. A mayor cantidad de iteraciones, más libertad tendrá el algoritmo de buscar nuevos vecindarios y lograr una mejor solución.

4. Experimentos Computacionales

Problema P43

Experimento 1:

- Población: 200
- Selección de Sobrevivientes: Reemplazo total
- Selección de Padres: Torneo de tamaño 2
- Operador de Cruce: Cruce de orden
- Operador de Mutación: Mutación por inversión con probabilidad de 0.4
- Fitness mejor solución: 6223

Experimento 2:

- Población: 200
- Selección de Sobrevivientes: Steady state con reemplazo de 175 individuos

- Selección de Padres: Torneo de tamaño 2
- Operador de Cruce: Cruce de orden
- Operador de Mutación: Mutación por inversión con probabilidad de 0.4
- Fitness mejor solución: 5625
- Solución: 12 14 13 33 34 31 32 10 8 11 9 7 5 6 30 29 28 27 3 1 2 36 37 4 0 35 22 21 23 24 26 25 38 39 42 40 41 15 16 20 18 19 17

Experimento 3:

- Población 300
- Selección de Sobrevivientes: Selección por ruleta
- Selección de Padres: Torneo de tamaño 2
- Operador de Cruce: Cruce de arcos
- Operador de Mutación: Mutación por inversión con probabilidad de 0.2
- Fitness mejor solución: 5621
- Solución: 26 25 21 22 23 24 15 17 16 18 19 20 12 14 13 33 34 31 32 11 10 9 8 7 6 5 30 29 28 27 3 1 2 36 35 0 4 37 39 38 42 41 40

Experimento 4:

- Población 300
- Selección de Sobrevivientes: Steady State de 275
- Selección de Padres: Torneo de tamaño 2
- Operador de Cruce: Orden
- Operador de Mutación: Mutación por inversión con probabilidad de 0.7
- Fitness mejor solución: 5620
- Solución: 13 12 34 33 31 32 9 11 10 8 6 7 5 30 29 28 27 1 2 3 36 37 39 38 42 41 40 26 25 24 21 23 22 35 0 4 17 18 19 20 15 16 14

Problema BR17

Experimento 1:

- Población: 100
- Selección de Sobrevivientes: Steady State con reemplazo de 75 individuos
- Selección de Padres: Torneo de tamaño 2
- Operador de Cruce: Cruce de orden
- Operador de Mutación: Mutación por inversión con probabilidad de 0.7
- Fitness mejor solución: 39
- Solución: 11 2 13 12 1 9 10 15 14 6 5 3 4 7 16 8 0

5. Resultados y conclusiones de cada problema

Problema P43

Este problema fue el foco de trabajo, ya que era el que mayor tamaño tenía (43 ciudades) y por ende mayor complejidad. Al empezar con este problema, nos encontramos con diferentes obstáculos. El obstáculo que nos hizo cambiar de foco fue el siguiente:

- Una solución podría ser representada por diferentes formas y podía estar repetida, por lo tanto necesitábamos garantizar que exista la diversidad y no caer en una población homogénea o un óptimo local.

Como queríamos ver la diversidad de la población, necesitábamos un indicador para cada iteración. Para llevarlo a cabo guardamos además de la mejor solución por iteración, la peor, y al finalizar la ejecución, en el archivo con todas las iteraciones imprimimos la diferencia entre el fitness de la mejor y la peor solución. Muchas veces veíamos que este arrojaba un valor cercano a cero, lo cual significaba que todas las soluciones tenían similar fitness, y que potencialmente eran muy parecidas o iguales. Al reconocer esto, mejoró mucho la curva de aprendizaje y por lo tanto la dirección de los experimentos cambió. A partir de entonces, priorizamos la diversidad e incorporamos cambios para disminuir la presión selectiva:

- Disminuimos el tamaño del torneo
- Evitamos el uso del operador de cruce basados en Arcos, ya que dos padres muy probablemente generarán un hijo igual a uno de los padres.
- En la selección de sobrevivientes, priorizamos usar Selección por torneo o Steady State guardando muy pocas soluciones óptimas locales.

Luego de estos cambios, empezamos a ver un avance en las soluciones y fuimos experimentando con diferentes configuraciones. En el indicador que habíamos agregado previamente, empezamos a ver valores más altos (mayor diferencia), lo cual significó que empezaba a haber mayor variedad en la población. Con este tipo de configuraciones fue que logramos que el algoritmo se estanque menos, y llegamos a mejores soluciones, entre ellas, la óptima conocida.

Problema BR17

Cuando hicimos la prueba de este problema, obtuvimos resultados exitosos desde los primeros experimentos. Esto se debe a que teníamos la experiencia de que obstáculos habíamos tenido antes y corregirlos a la primera. Decidimos usar una configuración parecida a la que usamos para conseguir la mejor solución en el problema anterior (P43).

6. Conclusiones generales

Entre las decisiones tomadas e implementadas durante el trabajo, optamos por usar un indicador (Diferencia) para estimar y visualizar la diversidad en la población. Si bien somos conscientes de que métricas como el “desvío estándar” ofrecerían mejor precisión, priorizamos una alternativa sencilla que nos brinda un panorama general. Adicionalmente, implementamos una condición de terminación distinta: Configuramos un límite de más de 20000 iteraciones junto a un fitness objetivo, si logra mejorarlo, detiene la simulación.

El objetivo de estos cambios fue poder observar el comportamiento de la población a largo plazo y poder evaluar si lograba una mejora continua o si quedaban atrapadas en óptimos locales.

Hacer este análisis, fue clave para poder evidenciar que el operador de cruce no puede cumplir su rol de manera efectiva, sin una diversidad correcta en la población y viceversa.

Finalmente, podemos concluir que los Algoritmos evolutivos tienen la capacidad de adaptación y su gran fuerte de escapar de óptimos locales, otorgándole a esta tecnología una gran variedad de aplicaciones. Si bien la implementación del código fue relativamente sencilla, lo más interesante fue relacionar los conceptos teóricos y analizar qué sucedía internamente de cada experimento.

7. Bibliografía

Freire, N. (2025, 4 de marzo). *Los tres puntos clave para entender la Teoría de la Evolución de Darwin*. National Geographic.

https://www.nationalgeographic.com.es/ciencia/tres-puntos-clave-para-entender-teoria-evolucion-darwin_21543